Lecture 2: Dart programming language

Outline

- Sets `var setOfNumber = [1,2,3,'Four','Five']; //`
- Maps
- Operator

Learning Outcome

- Understand the principals and use of set, map, and operator of Dart language
- Can create application using set and map
- Study and can use Dart programming language to implement mobile development

Dart programming language

- **Built-in Types in Dart**

The Dart language has special support for the following types:

- Numbers
- Strings
- Booleans
- Lists (also known as arrays)
- Sets
- Maps

Operators

- Runes (for expressing Unicode characters in a string)
- Symbols

Dart programming language

- **Set**

Program I

```
main() {
List fruitCollection = ['Mango', 'Apple', 'Jack fruit’];        // array
var myIntegers = [1, 2, 3, 'non-integer object’];               // set
print(myIntegers[3]);
print(fruitCollection[0]);
}
```

การเข้าถึงข้อมูลจะเหมือนกันเลย เริ่มขึ้นจาก 0

//output

non-integer object

Mango

1. Dart lists use **zero-based indexing** like all the other collections you may have seen in other programming languages.
2. Just think of a list as a key-value pair, where **0 is the index of the first value or element.**

Dart programming language

- **Get, Set, Go**

There are small differences in syntax between **List** and **Set**.

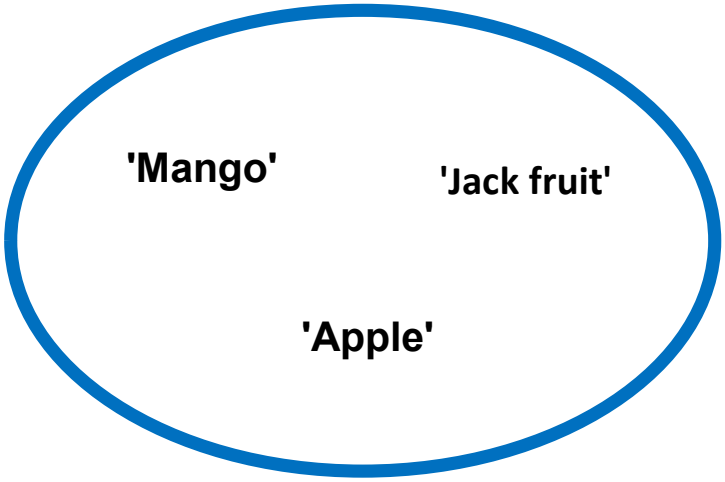
set จะมีฟังก์ชันใช้จัดการข้อมูลมากกว่า List(Array) ----

```
main() {
var fruitCollection = {'Mango', 'Apple', 'Jack fruit'};
print(fruitCollection.lookup('Apple'));
}
```

หา 'Apple'---

//output

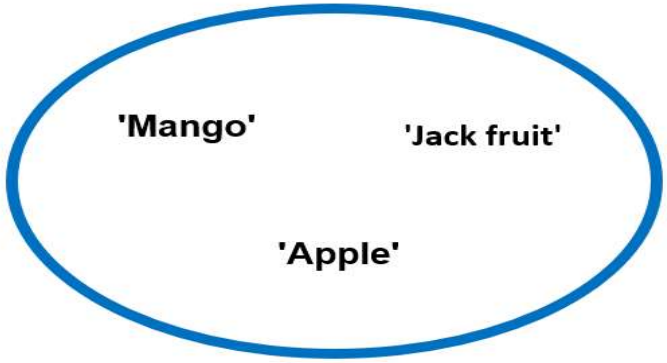
Apple



Dart programming language

- **Get, Set, Go**

There are small differences in syntax between **List** and **Set**.




```
main() {
var fruitCollection = {'Mango', 'Apple', 'Jack fruit'};
print(fruitCollection.lookup('Apple'));
}
```

//output

Apple

ใช้ Method .lookup() เพื่อหาบางอย่าง ถ้าไม่เจอก็จะ return กลับมาว่า Null-

We can search a set using the lookup() method. If we search for something else, it returns null.

```
main() {
var fruitCollection = {'Mango', 'Apple', 'Jack fruit'};
print(fruitCollection.lookup('Something Else'));
}
```

//output

null

จาก array ---> Set ---> Maps

Dart programming language

- **Map** = Dictionary ---

When we write the following, it does not create a Set, but **a Map:**

```
var myInteger = {};
```

```
main() {
var myInteger = {};
if(myInteger.isEmpty){      เช็คว่า Maps มันว่างหรือไม่--
print("It is a map that has no key, value pair.");
} else print("It is a set that has no key, value pair.");
}
```

//output

It is a map that has no key, value pair.

The literal **{}** is a default to **the Map type**

This is a map that has no key-value pair. It means the map is empty.

We will see lots of examples of **sets** in the future, while we build our mobile application.

In general, **a map** is **an object** that associates **keys with values**. มี Keys และ value --

Dart programming language

- **Map**

When we write the following, it does not create a Set, but a Map:

```
var myInteger = {};
////////////////////////////////////////////////////////////////
main(List<String> arguments) {
var myInteger = {};
if(myInteger.isEmpty){
print("It is a map that has no key, value pair.");
} else print("It is a set that has no key, value pair.");
}
```

//output

It is a map that has no key, value pair.

- The literal **{}** is a default to **the Map type**
- This is a map that has no key-value pair. It means the map is empty.
- We will see lots of examples of **sets** in the future, while we build our mobile application.
- In general, **a map** is an object that associates **keys with values**.
- The **set** has also **keys**, but they are **implicit**. In cases of sets, we call them **indexes.**

Dart programming language

- **Map**

Let’s see one example of **the Map type** by mapping literals. While writing **keys and values**, it is important to note that **each key occurs only once**, but **you can use the same value many times**. Key 1 อันจะเก็บ value 1 ค่า--

```
////////////////////////////////////////////////////////////////
main() {
var myProducts = {
'first' : 'TV',
'second' : 'Refrigerator',
'third' : 'Mobile',
'fourth' : 'Tablet',
'fifth' : 'Computer'
};
print(myProducts['third']);
}
```

ใช้ ['Key'] ในการเข้าถึงค่า -

//output

Mobile

Dart programming language

- **Map**

Dart understands that myProducts has the type **Map<String, String>** (Map <Key, Value>);
we could have made **the key integers or numbers**, instead of **a string type**.

```
////////////////////////////////////////////////////////////////
main() {
var myProducts = {
'first' : 'TV',
'second' : 'Refrigerator',
'third' : 'Mobile',
'fourth' : 'Tablet',
'fifth' : 'Computer'
};
print(myProducts['third']);
}
////////////////////////////////////////////////////////////
main() {
var myProducts = {
1 : 'TV',
2 : 'Refrigerator',
3 : 'Mobile',
4 : 'Tablet',
5 : 'Computer'
};
print(myProducts[3]);
}
```

กรณีที่ Key เป็นค่า int ก็พิมพ์ตอนเรียกใช้ได้ [key] (ไม่ต้องใส่ Q)---

Dart programming language

- **Map**

Can we add **a Set type** collection of values inside **a Map**?

```
//////////////////////////////////////////////////////////////
main(List<String> arguments) {
var myProducts = {
'first' : 'TV',
'second' : 'Refrigerator',
'third' : 'Mobile',
'fourth' : 'Tablet',
'fifth' : 'Computer'
};
print(myProducts['third']);
}
//////////////////////////////////////////////////////////////
main() {
var myProducts = {
1 : 'TV',
2 : 'Refrigerator',
3 : 'Mobile',
4 : 'Tablet',
5 : 'Computer'
};
print(myProducts[3]);
}
```

```
main() {
Set mySet = {1, 2, 3};
var myProducts = {
1 : 'TV',
2 : 'Refrigerator',
3 : mySet.lookup(2),
4 : 'Tablet',
5 : 'Computer'
};
print(myProducts[3]);
}
```

ค่าสามารถมาจาก lookup ได้-

Dart programming language

- **Map**

Can we add **a Set type** collection of values inside **a Map**?

```
//////////////////////////////////////////////////////////////
main(List<String> arguments) {
var myProducts = {
'first' : 'TV',
'second' : 'Refrigerator',
'third' : 'Mobile',
'fourth' : 'Tablet',
'fifth' : 'Computer'
};
print(myProducts['third']);
}
//////////////////////////////////////////////////////////////
main(List<String> arguments) {
var myProducts = {
1 : 'TV',
2 : 'Refrigerator',
3 : 'Mobile',
4 : 'Tablet',
5 : 'Computer'
};
print(myProducts[3]);
}
```

```
main() {
Set mySet = {1, 2, 3};
var myProducts = {
1 : 'TV',
2 : 'Refrigerator',
3 : mySet.lookup(2),
4 : 'Tablet',
5 : 'Computer'
};
print(myProducts[3]);
}
```

We injected **a collection of the Set type**, and we also **looked up** the defining value through the Map key. Here, inside **the Map key-value pair**, we have added the set element number 2 in this way: 3 : **mySet.lookup(2)**

Dart programming language

- **Map**

Can we add **a Set type** collection of values inside **a Map**?

```
main() {
Set mySet = {1, 2, 3};
var myProducts = {
1 : 'TV',
2 : 'Refrigerator',
3 : mySet.lookup(2),
4 : 'Tablet',
5 : 'Computer'
};
print(myProducts[3]);
}

//Output
2
```

Dart programming language

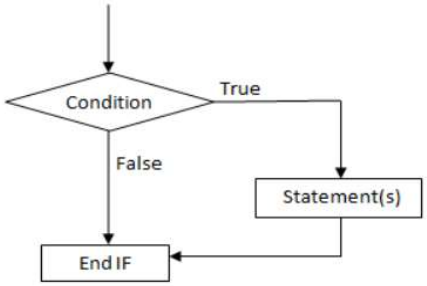
- **Map**

You can create the same products list by using the **Map constructor**. For beginners, the term constructor might seem difficult

กำหนดค่าตายตัวไม่สามารถแก้ไขได้

```
main() {
var myProducts = {
'first' : 'TV',
'second' : 'Refrigerator',
'third' : 'Mobile',
'fourth' : 'Tablet',
'fifth' : 'Computer'
};

print(myProducts['third']);
}
```



if-then

Run ตอนแรก --

```
main() {
```

ประกาศจองพื้นที่ไว้ก่อน

```
var myProducts = Map();
```

สามารถแก้ไขได้ ---

ใส่ค่าทีหลังได้ ---

```
myProducts['first'] ='TV';
myProducts['second'] ='Mobile';
myProducts['third'] ='Refrigerator';
```

นี่คือการเขียน if-then คือเขียนแค่ if เฉย ๆ ไม่ต้องมี else --

```
if(myProducts.containsValue('Mobile')){
print("Our products list has
${myProducts['second']}");
}
}
```

Dart programming language

- **Map**

You can create the same products list by using the **Map constructor**. For beginners, the term constructor might seem difficult

```
main(List<String> arguments) {

var myProducts = {
'first' : 'TV',
'second' : 'Refrigerator',
'third' : 'Mobile',
'fourth' : 'Tablet',
'fifth' : 'Computer'
};
```

ค้นหาด้วย Key

```
print(myProducts['third']);
}
```

```
main() {

var myProducts = Map();
myProducts['first'] ='TV';
myProducts['second'] ='Mobile';
myProducts['third'] ='Refrigerator';
```

มีค่า .... อยู่ไหม

ค้นหาด้วย Value --

```
if(myProducts.containsValue('Mobile')){
print("Our products list has
${myProducts['second']}");
}
}
//output
Our products list has Mobile
```

Dart programming language

- **Operators**

หาเศษ --

The usual arithmetic operators are : **add (+)**, **subtract (-), multiply (*), divide (/)**, and **modulo or remainder (%)**; a special operator, divide**, returning an integer looks like this: ~/.**

หารแบบไม่เอาทศนิยม

```
main() {
int aNum = 12;
double aDouble = 2.25;
var theResult = aNum ~/ aDouble;
print(theResult);
}

//output
5
```

หารแบบไม่เอาทศนิยม

Dart programming language

- **Operators**

The usual arithmetic operators are : **add (+), subtract (-), multiply (*), divide (/), and modulo or remainder (%);** a special operator, divide, **returning an integer looks like this: ~/.**

เปรียบเทียบการหาร 2 แบบ ---

```
main() {
int aNum = 12;
double aDouble = 2.25;
var theResult = aNum ~/ aDouble;
print(theResult);
}

//output
5
```

```
main() {
int aNum = 12;
double aDouble = 2.25;
var theResult = aNum / aDouble;
print(theResult);
}

//output
5.333333333333333
```

แบบธรรมดาจะมีเศษ

Dart programming language

- **Operators**

- One key feature of Dart is that it supports both **prefix and postfix** increment and decrement operators.

Here, **a prefix means ++variable or --variable**. เพิ่มค่าก่อน ---

- These either add 1 or subtract 1 from the variable value, respectively.

The postfix does the same; only the syntax changes, like this: **variable++ or variable--**. เพิ่มค่าหลัง ---

```
main() {
int aNum = 12;
aNum++;   เพิ่มค่าไป 1
print(aNum);
++aNum;   เพิ่มค่าไป 1
print(aNum);
int anotherNum = aNum + 1;
print(anotherNum);
}
```

Dart programming language

- **Operators**

- One key feature of Dart is that it supports both **prefix and postfix** increment and decrement operators.

Here, **a prefix means ++variable or --variable**.

- These either add 1 or subtract 1 from the variable value, respectively.

The postfix does the same; only the syntax changes, like this: **variable++ or variable--**.

```
main() {
int aNum = 12;
aNum++;
print(aNum);
++aNum;
print(aNum);
int anotherNum = aNum + 1;
print(anotherNum);
}
```

```
//output
13
14
15
```

Dart programming language

- **Operators**

- One key feature of Dart is that it supports both **prefix and postfix** increment and decrement operators.

Here, **a prefix means ++variable or --variable**.

- These either add 1 or subtract 1 from the variable value, respectively.

The postfix does the same; only the syntax changes, like this: **variable++ or variable--**.

ไม่มีปัญหาจ้า ---

```
main() {
int aNum = 12;
aNum++;
++aNum;
int anotherNum = aNum + 1;
print(anotherNum);
}
//output
15
```

แต่มีจะปัญหากับอันนี้ --

```
main() {
int aNum = 12;
```

จะบวก 1 ก่อนสมาการคำนวณ --

```
int anotherNum = ++aNum + 1;
```

การใส่ pre pos ในสมาการ --

```
print(anotherNum);
}

//Output
14
```

Dart programming language

- **Operators**

- One key feature of Dart is that it supports both **prefix and postfix** increment and decrement operators.

Here, **a prefix means ++variable or --variable**.

- These either add 1 or subtract 1 from the variable value, respectively.

The postfix does the same; only the syntax changes, like this: **variable++ or variable--**.

```
main() {
int aNum = 12;
aNum++;
++aNum;
int anotherNum = aNum + 1;
print(anotherNum);
}
//output
15
```

```
main() {
int aNum = 12;
                          บวก 1 หลังการคำนวณสมาการ ---
int anotherNum = aNum++ + 1;
print(anotherNum);        ก็คือ 12 + 1 = 13
}

                          แล้วภายหลังจะไปเพิ่มค่า
                          +1 ภายหลัง --

//Output
13

แล้วเมื่อไรมันจะเป็น 14
```

Dart programming language

- **Operators : Relational Operators** == เท่ากับ

- Relational operators are also called **equality operators because ==** means “equal,” and other **relational operators usually check for equality** in various forms.

```
main() {
int firstNum = 40;
int secondNum = 41;
if (firstNum != secondNum){
print("$firstNum is not equal to the $secondNum");
} else print("$firstNum is equal to the $secondNum");
}
//output
40 is not equal to the 41
```

ไม่เท่ากับ --

Dart programming language

- **Operators : Relational Operators**

- Relational operators are also called **equality operators because ==** means “equal,” and other **relational operators usually check for equality** in various forms.

```
main() {
int firstNum = 40;
int secondNum = 41;
if (firstNum != secondNum){
print("$firstNum is not equal to the $secondNum");
} else print("$firstNum is equal to the $secondNum");
}
```

//output

40 is not equal to the 41

```
main() {
int firstNum = 40;
int secondNum = 40;
if (firstNum == secondNum){
print("$firstNum is equal to the $secondNum");
} else print("$firstNum is not equal to the
$secondNum");
}
```

//output

40 is equal to the 40

Dart programming language

- **Operators : Relational Operators**

- "If firstNum equals secondNum OR if thirdNum equals fourthNum,
- We use the **OR (||)** operator to implement this logic.
- This is not the case for the **AND (&&)** relational operator

```
main() {
int firstNum = 40;
int secondNum = 40;
if (firstNum == secondNum){
print("$firstNum is equal to the $secondNum");
} else print("$firstNum is not equal to the
$secondNum");
}
```

```
main() {
int firstNum = 40;
int secondNum = 40;
int thirdNum = 74;
int fourthNum = 56;
if (firstNum == secondNum || thirdNum == fourthNum){
print("If choice between 'true' or 'false', the 'true' gets
the precedence.");
} else print("If choice between 'true' or 'false', the
'false' gets the precedence.");
}
```

//output

```
If choice between 'true' or 'false', the 'true' gets the
precedence.
```

Dart programming language

- **Operators : Relational Operators**

- "If firstNum equals secondNum OR if thirdNum equals fourthNum,
- We use the **OR (||)** operator to implement this logic.
- This is not the case for the **AND (&&)** relational operator

```
main() {
int firstNum = 40;
int secondNum = 40;
int thirdNum = 74;
int fourthNum = 56;
if (firstNum == secondNum || thirdNum == fourthNum)
{
print("If choice between 'true' or 'false', the 'true' gets
the precedence.");
} else print("If choice between 'true' or 'false', the
'false' gets the precedence.");
}
```

//output

```
If choice between 'true' or 'false', the 'true' gets the
precedence.
```

```
main() {
int firstNum = 40;
int secondNum = 40;
int thirdNum = 74;
int fourthNum = 56;
if (firstNum == secondNum && thirdNum == fourthNum)
{
print("If choice between 'true' or 'false', in this case
the 'true' gets the precedence.");
} else print("If choice between 'true' or 'false', in this
case the 'false' gets the precedence.");
}
```

//output

```
If choice between 'true' or 'false', in this case the 'false'
gets the precedence.
```

Dart programming language

- **Operators : Relational Operators**

- "If firstNum equals secondNum OR if thirdNum equals fourthNum,
- We use the **OR (||)** operator to implement this logic.
- This is not the case for the **AND (&&)** relational operator

```
main() {
int aNUmber = 35;
if(!(aNUmber != 150) && aNUmber <= 150)
{
print("It's true");
} else print("It's false.");
}
```

Can you guess what the output would be?
The first statement is false because we have negated a true statement by using the ! sign.
!(aNUmber != 150)

เติมนิเศษ ได้ด้วย ---

Dart programming language

- **Operators : Relational Operators**

- "If firstNum equals secondNum OR if thirdNum equals fourthNum,
- We use the **OR (||)** operator to implement this logic.
- This is not the case for the **AND (&&)** relational operator

```
main() {
int aNUmber = 35;
if(!(aNUmber != 150) && aNUmber <= 150)
{
print("It's true");
} else print("It's false.");
}
```

Can you guess what the output would be?
The first statement is false because we have negated a true statement by using the ! sign.
!(aNUmber != 150)

The second statement is true; the value is less than or equal to 150.
aNUmber <= 150

Dart programming language

- **Operators : Relational Operators**

- "If firstNum equals secondNum OR if thirdNum equals fourthNum,
- We use the **OR (||)** operator to implement this logic.
- This is not the case for the **AND (&&)** relational operator

```
main() {
int aNUmber = 35;
if(!(aNUmber != 150) && aNUmber <= 150)
{
print("It's true");
} else print("It's false.");
}
```

Can you guess what the output would be?
The first statement is false because we have negated a true statement by using the ! sign.
!(aNUmber != 150)

The second statement is true; the value is less than or equal to 150.
aNUmber <= 150

Since the logical operator is AND (&&) here, the whole expression will be **false**.

!(aNUmber != 150) && aNUmber <= 150

Dart programming language

- **Operators : Relational Operators**

- "If firstNum equals secondNum OR if thirdNum equals fourthNum,
- We use the **OR (||)** operator to implement this logic.
- This is not the case for the **AND (&&)** relational operator

```
main() {
int aNUmber = 35;
if(!(aNUmber != 150) && aNUmber <= 150)
{
print("It's true");
} else print("It's false.");
}
```

We can use **> for greater than**, or use **< for less than**

Can you guess what the output would be?
The first statement is false because we have negated a true statement by using the ! sign.
!(aNUmber != 150)

The second statement is true; the value is less than or equal to 150.
aNUmber <= 150

Since the logical operator is AND (&&) here, the whole expression will be false.

!(aNUmber != 150) && aNUmber <= 150

Dart programming language

- **Operators : Type Test Operators**

```
main() {
int myNumber = 13;
print(myNumber is int);
print(myNumber is! int);
print(myNumber is! bool);
print(myNumber is bool);
}
```

ใช้ is ได้เพื่อเช็คว่า ใช่ใช่ไหม ---

ใช้ is! ได้

//output

```
true
false
true
false
```

Dart programming language

- **Operators : Type Test Operators**

```
main() {
int firstNum = 10;
int secondNum=20;
print(secondNum);
print("After using an assignment operator, the value changes.");
secondNum += secondNum;     secondNum = secondNum + secondNum
print(secondNum);
print("After using an assignment operator, the value changes again.");
secondNum -= secondNum;       num -= 10 ==> num = num - 10
print(secondNum);
if (secondNum == null) print("It is true.");
}
```

//output

20

After using an assignment operator, the value changes.

40

After using an assignment operator, the value changes again.

0

Summary

We need to remember a few things, such as the following:

```
var isValid = true;
```

- **var is the data type.**
- **isValid is the variable name (or spot in memory).**
- **true is a literal.**

You can mention the data type of a variable as int, double, String

Activity 2.1

1. Write an application using a Dart language that shows your first name, family name, major, and university. You use any String types of Dart language.

2. Write an application using a Dart language that shows your student code (key), first name, family name, major, and university (value). You use the **Map type of** Dart language as Map <Key, Value>. Use command to append your information and four friends into Map <Key, Value>.

https://dartpad.dartlang.org.

Dart programming language

- **Get, Set, Go**

Can we add **a Set type** collection of values inside **a Map**?

```
//////////////////////////////////////////////////////////////
main(List<String> arguments) {
var myProducts = {
'first' : 'TV',
'second' : 'Refrigerator',
'third' : 'Mobile',
'fourth' : 'Tablet',
'fifth' : 'Computer'
};
print(myProducts['third']);
}
//////////////////////////////////////////////////////////////
main(List<String> arguments) {
var myProducts = {
1 : 'TV',
2 : 'Refrigerator',
3 : 'Mobile',
4 : 'Tablet',
5 : 'Computer'
};
print(myProducts[3]);
}
```

```
main(List<String> arguments) {
Set mySet = {1, 2, 3};
var myProducts = {
1 : 'TV',
2 : 'Refrigerator',
3 : mySet.lookup(2),
4 : 'Tablet',
5 : 'Computer'
};
print(myProducts[3]);
}
```